

CV8: ViewModel a oddelenie UI od logiky v miniaplikácii BMI Calculator - Teória

1 Cieľ cvičenia

Cieľom tohto cvičenia je naučiť sa pracovať s triedou `ViewModel` v prostredí Jetpack Compose. V predchádzajúcich cvičeniach bol stav obrazovky vytváraný priamo v composable funkciách. Takýto prístup je vhodný pri prvých ukážkach, ale pri väčších obrazovkách začína byť kód menej prehľadný. V tomto cvičení sa preto logika aplikácie presunie mimo používateľského rozhrania a bude spracovaná vo `ViewModel`.

Výsledkom bude samostatná miniaplikácia `BMI Calculator`, v ktorej používateľ zadá výšku a hmotnosť, stlačí tlačidlo a aplikácia vypočíta BMI, určí kategóriu výsledku a zobrazí ju na výsledkovej karte. Súčasťou prezentácie výsledku bude aj obrázok uložený lokálne v projekte.

2 Prečo použiť ViewModel

Trieda `ViewModel` slúži ako držiteľ stavu obrazovky a logiky, ktorá s týmto stavom pracuje. Composable funkcie majú v ideálnom prípade najmä zobrazovať dáta a odovzdávať používateľské akcie ďalej. Výpočty, validácia vstupov a rozhodovanie o tom, aký výsledok sa má zobrazíť, patria do samostatnej vrstvy.

Takéto rozdelenie prináša viacero výhod. Kód sa lepšie číta, pretože používateľské rozhranie nie je premiešané s výpočtami. Funkcie vo `ViewModel` sa dajú ľahšie testovať. Obrazovka je prehľadnejšia, pretože dostáva pripravený stav a iba ho zobrazuje. Tento prístup je zároveň prirodzeným prechodom k ďalším témam, ako sú perzistencia údajov alebo architektúra aplikácie.

3 Architektúra miniaplikácie

Aplikácia bude rozdelená na niekoľko menších častí. Trieda `MainActivity` bude mať len minimálnu úlohu. Vytvorí Compose obsah a zobrazí hlavnú obrazovku. Samotná obrazovka bude definovaná v samostatnom súbore a bude prijímať stav a callback funkcie. Trieda `BmiViewModel` bude držať vstupy, výsledok a funkcie na spracovanie udalostí. Dáta obrazovky budú reprezentované triedou `BmiUiState`.

Tok údajov bude jednosmerný. Používateľ zadá text do poľa. Obrazovka zavolá príslušnú funkciu vo `ViewModel`. `ViewModel` aktualizuje `BmiUiState`. Compose zaregistruje zmenu stavu a automaticky prekreslí používateľské rozhranie.

4 Odporúčaná štruktúra súborov

Aby bolo rozdelenie zodpovedností jasné, je vhodné pripraviť projekt s jednoduchou štruktúrou. V tomto cvičení sa predpokladajú tieto základné súbory:

```
MainActivity.kt  
BmiScreen.kt  
BmiViewModel.kt  
BmiUiState.kt
```

Obrázky sa budú nachádzať v priečinku zdrojov projektu:

```
app/src/main/res/drawable/
```

Takéto členenie už teraz pomáha udržať projekt prehľadný. Pri otvorení projektu je jasné, kde sa nachádza aktivita, kde obrazovka, kde stav a kde logika.

5 Potrebná dependency

Ak projekt ešte neobsahuje podporu pre používanie `ViewModel` v Compose, je potrebné doplniť do súboru `build.gradle.kts` nasledujúcu dependency:

```
implementation("androidx.lifecycle:lifecycle-viewmodel-compose  
:2.10.0")
```

Táto knižnica umožní použiť funkciu `viewModel()` priamo v composable časti aplikácie.

6 Model stavu obrazovky

Namiesto viacerých samostatných premenných je vhodné vytvoriť jednu dátovú triedu, ktorá opisuje celý stav obrazovky. V tomto prípade bude obsahovať text zadaný vo vstupných poliach, vypočítané BMI, kategóriu výsledku, správu pre používateľa a prípadnú chybovú hlášku.

```
data class BmiUiState(  
    val heightInput: String = "",  
    val weightInput: String = "",  
    val bmiValue: String = "",  
    val category: BmiCategory? = null,  
    val resultMessage: String = "",  
    val errorMessage: String? = null,  
    val showResult: Boolean = false  
)
```

Tento prístup je dôležitý preto, že obrazovka pracuje s jedným uceleným stavom. Je tak jasné, aké dáta môže UI zobrazovať.

7 Prečo sa stav mení cez copy

Keďže `BmiUiState` je dátová trieda, pri zmene jednej hodnoty je vhodné vytvoriť novú verziu stavu pomocou funkcie `copy`. Tým sa zachovajú ostatné položky a zmenia sa len tie, ktoré sú potrebné.

```
uiState = uiState.copy(  
    heightInput = newValue,  
    errorMessage = null  
)
```

Tento štýl je v Compose bežný. Pomáha udržať zmeny stavu čitateľné a predvídateľné.

8 Kategórie BMI

Výsledok nebude zobrazovaný iba ako číslo. Aplikácia určí aj kategóriu. Na tento účel je vhodné vytvoriť enum triedu:

```
enum class BmiCategory {  
    UNDERWEIGHT,  
    NORMAL,  
    OVERWEIGHT,  
    OBESITY  
}
```

Túto hodnotu vypočíta `ViewModel` a používateľské rozhranie ju neskôr použije pri výbere farby a obrázka.

9 Logika vo ViewModel

ViewModel bude obsahovať stav a funkcie, ktoré spracúvajú používateľské akcie. Obrazovka nebude sama počítat BMI. Namiesto toho iba pošle informáciu o tom, že používateľ klikol na tlačidlo.

```
class BmiViewModel : ViewModel() {

    var uiState by mutableStateOf(BmiUiState())
    private set

    fun onHeightChange(newValue: String) {
        uiState = uiState.copy(
            heightInput = sanitizeNumberInput(newValue),
            errorMessage = null
        )
    }

    fun onWeightChange(newValue: String) {
        uiState = uiState.copy(
            weightInput = sanitizeNumberInput(newValue),
            errorMessage = null
        )
    }

    fun calculateBmi() {
        // validation and BMI calculation
    }

    fun reset() {
        uiState = BmiUiState()
    }
}
```

Týmto spôsobom sa všetka rozhodovacia logika nachádza mimo composable funkcií.

10 Prečo mutableStateOf funguje aj vo ViewModel

Vlastnosť `uiState` je uložená pomocou `mutableStateOf`. To znamená, že Compose vie sledovať jej zmeny. Ak sa hodnota zmení, composable funkcie, ktoré ju používajú, sa znovu prekreslia. Vďaka tomu nie je potrebné ručne oznamovať obrazovke, že sa má obnoviť.

To je dôležitá myšlienka celého cvičenia. `ViewModel` mení stav a Compose automaticky reaguje na novú hodnotu.

11 Výpočet BMI

Samotný výpočet vychádza zo známeho vzorca, kde sa hmotnosť v kilogramoch delí druhou mocninou výšky v metroch. Keďže používateľ zadáva výšku v centimetroch, je najprv potrebné ju prepočítať na metre.

```
val heightMeters = heightCm / 100.0
val bmi = weightKg / heightMeters.pow(2.0)
```

Vo `ViewModel` je zároveň vhodné vykonať aj validáciu. Ak sú polia prázdne, ak obsahujú neplatný text alebo ak používateľ zadal nereálne hodnoty, `ViewModel` nastaví chybovú správu a výsledok sa nezobrazí.

12 Validácia vstupov

Pri formulároch nestačí iba načítať text. Je potrebné skontrolovať, či je vstup použiteľný pre výpočet. V tomto cvičení je vhodné overiť, či používateľ zadal obe hodnoty, či sú to čísla a či sa pohybujú v realistickom rozsahu.

Takáto validácia patrí do `ViewModel`, nie priamo do composable funkcie. Obrazovka len zobrazí chybovú správu, ktorú dostane v stave.

13 Mapovanie kategórie na vzhľad

Dôležitou myšlienkou tohto cvičenia je rozdelenie zodpovednosti medzi logiku a UI. ViewModel určí, do akej kategórie výsledok patrí. Používateľské rozhranie potom podľa kategórie vyberie správnu farbu a správny obrázok. To znamená, že ViewModel nevracia konkrétny Color ani konkrétny obrázok.

```
private fun BmiCategory.presentation(): CategoryPresentation {
    return when (this) {
        BmiCategory.UNDERWEIGHT -> CategoryPresentation(
            label = "Underweight",
            backgroundColor = Color(0xFFFFF3CD),
            textColor = Color(0xFF6A4F00),
            imageRes = R.drawable.bmi_underweight
        )
        BmiCategory.NORMAL -> CategoryPresentation(
            label = "Normal weight",
            backgroundColor = Color(0xFFDFF5E1),
            textColor = Color(0xFF1B5E20),
            imageRes = R.drawable.bmi_normal
        )
        BmiCategory.OVERWEIGHT -> CategoryPresentation(
            label = "Overweight",
            backgroundColor = Color(0xFFFFE0B2),
            textColor = Color(0xFF7A4B00),
            imageRes = R.drawable.bmi_overweight
        )
        BmiCategory.OBESITY -> CategoryPresentation(
            label = "Obesity",
            backgroundColor = Color(0xFFFF8D7DA),
            textColor = Color(0xFF8A1C1C),
            imageRes = R.drawable.bmi_obesity
        )
    }
}
```

Takéto riešenie je technicky správne, pretože logická vrstva rozhoduje o stave a prezentačná vrstva rozhoduje o vzhľade.

14 Zobrazenie lokálnych obrázkov

Obrázky nebudú načítavané z internetu. V tomto cvičení je vhodnejšie uložiť ich priamo do projektu. Naučíte sa tak pracovať s lokálnymi zdrojmi bez toho, aby bolo potrebné zavádzať ďalšie knižnice na sieťové načítanie obrázkov.

Obrázky je potrebné uložiť do priečinka:

```
app/src/main/res/drawable/
```

Názvy obrázkov v tomto cvičení budú:

```
bmi_underweight.png  
bmi_normal.png  
bmi_overweight.png  
bmi_obesity.png
```

V Compose sa následne zobrazia pomocou funkcie `painterResource`:

```
Image(  
    painter = painterResource(id = R.drawable.bmi_normal),  
    contentDescription = "Normal weight"  
)
```

Tento princíp je dôležitý aj v ďalších projektoch, pretože práca s `drawable` zdrojmi patrí medzi základné zručnosti pri tvorbe Android rozhrania.

15 Úloha MainActivity

V tomto cvičení má byť `MainActivity` čo najjednoduchšia. Nemá obsahovať výpočet BMI, validáciu vstupov ani rozhodovanie o tom, čo sa má zobrazíť. Jej úlohou je iba vytvoriť obrazovku a získať inštanciu `ViewModel`.

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContent {  
            MaterialTheme {  
                Surface {  
                    val bmiViewModel: BmiViewModel = viewModel()  
                    BmiScreen(  
                        uiState = bmiViewModel.uiState,  
                        onHeightChange = bmiViewModel::  
                            onHeightChange,  
                        onWeightChange = bmiViewModel::  
                            onWeightChange,  
                        onCalculateClick = bmiViewModel::  
                            calculateBmi,  
                        onResetClick = bmiViewModel::reset  
                    )  
                }  
            }  
        }  
    }  
}
```

Toto je hlavná idea celého cvičenia. `MainActivity` zostáva krátka a prehľadná.

16 Používateľské rozhranie

Samotná obrazovka môže byť rozdelená na menšie composable funkcie, napríklad hlavička, vstupná karta, výsledková karta a karta s chybovou správou. Takéto členenie pomáha udržať kód čitateľný. Každá časť má jasnú zodpovednosť a dá sa samostatne upravovať alebo použiť v `@Preview`.

Vo vstupnej časti budú dve textové polia a dve tlačidlá. Vo výsledkovej časti sa zobrazí číselná hodnota BMI, text kategórie, obrázok a krátka správa. Chybová karta sa zobrazí iba v prípade neplatného vstupu.

17 Použitie Preview

Aj pri cvičení s `ViewModel` je vhodné pripraviť aspoň jeden `@Preview`. Najjednoduchší prístup je vytvoriť preview s pripraveným testovacím `BmiUiState`, aby bolo možné rýchlo skontrolovať vzhľad obrazovky bez spustenia celej aplikácie.

```
@Preview(showBackground = true)
@Composable
fun BmiScreenPreview() {
    BmiScreen(
        uiState = BmiUiState(
            heightInput = "170",
            weightInput = "65",
            bmiValue = "22.5",
            category = BmiCategory.NORMAL,
            resultMessage = "Vysledok je v norme.",
            showResult = true
        ),
        onHeightChange = {},
        onWeightChange = {},
        onCalculateClick = {},
        onResetClick = {}
    )
}
```

Preview je vhodné aj preto, že pri práci s menšími composable časťami sa dá rýchlejšie kontrolovať rozloženie, texty a vizuálna prezentácia výsledku.